LAM Engineering Standards & Development Lifecycle

Mandatory engineering rules, milestone gates, manual test procedures, and performance tracking

Version 1.0 | Prepared 18 October 2026 | Applies to the standalone Learning Adaptation Model (LAM)

Purpose: establish an enforceable standard for design, implementation, manual verification, release, and ongoing operation of LAM.

1. Purpose and authority

This document defines the mandatory working standard for engineers building LAM (Learning Adaptation Model). It is an engineering operating standard, not merely a suggestion list. Requirements in this document must be converted into repository rules, code review checks, automated tests, manual test evidence, and release gates.

LAM is an independent, versioned learning-adaptation service. It consumes structured learning evidence and returns learner-state updates, diagnosis candidates, recommended next actions, and retrieval plans. Qrie and other host applications deliver the user experience and learning content.

- LAM owns: evidence ingestion and validation; event ledger; learner state; mastery estimates; diagnosis; adaptive decisions; assistance policy; retrieval scheduling; strategy-effectiveness evidence; session analysis; shared-model lifecycle.
- Qrie/host applications own: student UI, classroom rendering, lesson and content delivery, tutor conversations, voice/avatar, school workflows, and the delivery of the selected intervention.
- The shared model learns general patterns only from explicitly approved and governed data. Individual learner state remains separate. Live event ingestion must never silently train global model weights.

Authority rule: where this standard conflicts with a local implementation shortcut, the standard wins unless an approved, documented exception is recorded by the technical owner and security/privacy owner where applicable.

2. Non-negotiable engineering principles

– Correctness before complexity. Ship a deterministic, testable baseline before introducing complex ML, queues, agents, or distributed services.

– Evidence before inference. Every state change and recommendation must be traceable to validated evidence, a versioned policy, or an explicitly identified model.

– Unknown is not false. Missing, stale, invalid, or untrusted signals must remain unknown or be rejected; never silently coerce them into negative evidence.

– Assisted success is not independent mastery. Store assisted and unassisted performance separately. A correct answer after a hint or worked solution cannot by itself prove independent mastery.

– No forced diagnosis. A wrong answer is not automatically a misconception. Preserve competing hypotheses and ask for diagnostic evidence when necessary.

– Student isolation is mandatory. A student ID in a URL or request body is not authorization. Every data access must be authorized server-side and tenant-scoped.

– No hidden global learning. Do not train shared-model weights on each live event. Shared training must be a separate, approved, versioned offline pipeline.

– Reproducibility. Given the same normalized evidence, initial state, graph version, policy version, and model version, deterministic components must produce the same result.

– Version everything that changes behavior: API schemas, event schemas, learner-state schema, curriculum adapter, decision policies, scheduling algorithms, models, and evaluation datasets.

– Measure learning, not vanity metrics. Product effectiveness must be evaluated with independent mastery, delayed retention, transfer, and learning time—not only engagement, acceptance, or immediate correctness.

– Privacy by default. Minimize sensitive data, restrict logs, define retention/deletion behavior, and never use raw conversations for training by default.

– Fail safely. If evidence, a dependency, or a model is unavailable, return a defined safe fallback or explicit error. Never invent a confident recommendation.

3. Ownership boundaries and system contracts

Keep LAM modular and independently deployable. Prefer a modular monolith initially; do not create a separate network service for every subsystem without evidence that independent deployment is necessary.

- API layer: authentication/authorization, request validation, rate limits, idempotency keys, request IDs, error mapping.
- Evidence layer: canonical event schema, provenance, validation, deduplication, durable event ledger.
- Learner-state layer: versioned per-student/per-concept state, transactional updates, audit trail, state reconstruction where supported.
- Decision layer: diagnosis hypotheses, assistance policy, prerequisite-aware sequencing, retrieval scheduling, rationale and uncertainty.

- Shared-model pipeline: approved dataset preparation, offline training/evaluation, model registry, staged rollout, monitoring and rollback.
- Integration adapters: stable contracts to Qrie's concept graph and host applications; no imports of host application's internal packages.

Every recommendation should carry, where relevant: action type, target concept, assistance level, reason codes, evidence references, uncertainty, state version, policy version, graph version, model version, and a safe fallback.

4. Mandatory code and repository standards

4.1 Language and typing

- Use Python type hints for public functions and domain interfaces. Use Pydantic models for API boundaries and explicit domain types internally.
- Keep domain logic independent of FastAPI request objects, database sessions, and external providers.
- Validate inputs at boundaries; do not scatter ad-hoc validation through business logic.
- Avoid `Any`, untyped dictionaries, broad exception swallowing, and implicit conversions in critical state/decision paths. If unavoidable, document the reason and add tests.
- Use timezone-aware UTC timestamps. Define units and ranges for all numeric fields.

4.2 Architecture and maintainability

- Keep modules cohesive and dependencies one-directional: API → application services → domain logic; infrastructure implements domain-facing interfaces.
- Do not put business logic in route handlers, ORM hooks, background-job callbacks, or SDK wrappers.
- Do not duplicate the curriculum graph in LAM. Use a versioned adapter/contract.
- Avoid global mutable state, hidden side effects, circular imports, and implicit network calls inside domain logic.
- Every non-trivial algorithm must have a written specification, assumptions, input/output examples, and tests.

4.3 Database and event integrity

- Use migrations for every schema change; never rely on manual production database edits.
- Use transactions for event ingestion plus state update when they must be atomic.
- Enforce unique event/idempotency constraints in the database, not only in application memory.
- Use optimistic locking or an equivalent concurrency-safe mechanism for learner-state versions.
- Record event provenance, schema version, and the versions of policies/models that produced derived state.
- Never delete or rewrite historical evidence silently. Corrections must be represented and auditable.

4.4 API and error handling

- Maintain versioned HTTP/JSON contracts and a documented OpenAPI specification.
- Use consistent error shapes and stable error codes. Do not return stack traces or secrets to clients.
- Set explicit timeouts for downstream calls; retries must be bounded and safe for idempotent operations.
- Validate tenant and learner permissions on every endpoint, including administrative and background paths.
- Use pagination and bounded response sizes for histories and review lists.

4.5 Security and privacy

- Never commit credentials, tokens, private student data, production dumps, or secrets to the repository.
- Use least-privilege service credentials and separate development, test, staging, and production configuration.
- Do not log raw transcripts, student answers, access tokens, or sensitive payloads unless an approved, minimized use case requires it.
- Document retention, deletion, correction, consent, and training eligibility before enabling real student data.
- Treat all model outputs and external inputs as untrusted until validated against allowed schemas and policies.

4.6 Documentation

- Update docs with every behavior-changing feature: design decision, contract, configuration, migration, runbook, and known limitations.
- A pull request must explain why the change is needed, what changed, risks, tests run, manual checks, and rollback considerations.
- Do not describe a capability as complete unless it is implemented, tested, and evidenced.

5. Mandatory workflow for every change

– Create or update an issue with user/problem context, expected behavior, scope, out-of-scope items, acceptance criteria, privacy/security impact, and test plan.

– Write or update the design note for behavior-changing work. Identify contracts and invariants before coding.

– Create a small branch from the approved base. Keep one logical change per pull request where practical.

– Implement the smallest testable vertical slice. Add tests alongside code, not after the feature is considered finished.

– Run local quality gates: formatter, linter, type checker, unit tests, relevant integration tests, and security checks available in the repository.

– Perform manual tests using the documented scenario and record actual result, environment, build/commit, evidence, and defects.

– Review observability, failure behavior, migration safety, authorization, privacy, and rollback.

– Open a pull request with a completed checklist and linked test evidence. At least one reviewer other than the author must review behavior-changing work.

– Merge only when required CI checks pass, review comments are resolved, and acceptance criteria are satisfied.

– Deploy to a non-production environment first. Run smoke and regression checks, inspect logs/metrics, then promote according to the release gate.

– After release, verify the deployed version, key metrics, error rates, and a representative end-to-end learning flow. Roll back if a stop condition is met.

6. Required test strategy

6.1 Unit tests

Test pure functions and individual rules with explicit inputs and expected outputs. Cover normal cases, boundary values, invalid values, missing values, uncertainty, and failure paths.

Examples: mastery update for correct/incorrect responses; assistance-level transitions; diagnosis confidence thresholds; prerequisite eligibility; review interval changes; schema validation; reason-code generation.

6.2 Integration tests

Use a real test database or a faithful isolated database environment for transaction, migration, locking, idempotency, and persistence behavior. Do not use mocks as the only proof of database correctness.

Examples: event insert + state update transaction; duplicate event; simultaneous updates; rollback on partial failure; learner state retrieval after process restart; migration from the previous schema.

6.3 Contract tests

Verify request/response schemas, error codes, auth behavior, pagination, and compatibility for the Python SDK, TypeScript client, and Qrie integration adapter.

6.4 Security and privacy tests

Examples: Student A cannot read or modify Student B; tenant A cannot access tenant B; missing credentials are rejected; expired/invalid credentials fail; secrets are not present in logs; deletion and retention workflows behave as specified.

6.5 End-to-end tests

Exercise the whole loop: record an event → update state → request recommendation → host delivers intervention → record outcome → update state → schedule review → submit delayed assessment.

6.6 Model and policy evaluation

Use fixed benchmark scenarios and versioned evaluation datasets. Compare candidate policies/models to the current baseline. Report outcome metrics, uncertainty, subgroup behavior, regressions, and failure cases. No model should be promoted based only on training-set performance.

6.7 Manual testing

Manual testing is mandatory for each milestone and release, but it supplements automated tests rather than replacing them. Record the exact commit/build, configuration, scenario, expected result, actual result, evidence, and defect IDs.

7. Manual test procedure and evidence standard

7.1 Required manual test record

- Test ID and title.
- Requirement/issue ID and severity if it fails.
- Commit SHA, build version, environment, database migration version, policy/model version.
- Preconditions and seeded data; use synthetic student data unless approved otherwise.
- Exact steps, expected result, actual result, pass/fail/blocked status.
- Evidence: redacted logs, screenshots where useful, request/response IDs, database assertions, and metric snapshot.
- Tester, date/time, defect link, retest result, and sign-off.

7.2 Manual scenarios required before M2 integration

– Fresh learner: submit first event; verify profile/state initialization and evidence provenance.

– Correct independent answer: verify state changes according to the documented baseline and no assistance is recorded.

– Correct answer after a hint: verify assisted evidence is recorded and independent mastery is not promoted solely by this event.

– Wrong answer with ambiguous cause: verify diagnosis remains uncertain or requests a diagnostic question.

– Repeated failure: verify the policy changes intervention or follows its documented escalation limit.

– Prerequisite gap: verify the recommendation respects the graph and does not incorrectly skip a hard prerequisite.

– Duplicate event: resend the same event ID; verify no duplicate logical state update.

– Concurrent events: submit two events for the same learner/concept concurrently; verify no lost update and a valid state version.

– Cross-tenant access: attempt to read/write another tenant's learner; verify denial and no data leakage.

– Delayed review: submit a later independent assessment; verify state and review schedule update according to policy.

– Dependency outage: make the graph adapter or optional model unavailable; verify bounded failure and safe fallback.

– Restart/recovery: restart the service and verify learner state and event history persist.

7.3 Defect severity

- P0/Critical: cross-tenant data exposure, unauthorized access, corrupted learner state, unsafe data loss, or a release-blocking system outage. Stop release.
- P1/High: core learning loop incorrect, assisted success counted as independent mastery, duplicate updates, invalid prerequisite decisions, or major data integrity failure. Block release.
- P2/Medium: non-critical behavior mismatch with a documented workaround; release requires explicit owner approval and a tracked fix.
- P3/Low: cosmetic/documentation issue with no material effect on correctness, privacy, or learning outcomes; track in backlog.

8. Development lifecycle and milestone gates

M0 — Foundation and contracts

Build repository structure, configuration, authentication/authorization, tenant model, typed event/state/recommendation schemas, PostgreSQL migrations, event ledger, API skeleton, SDK skeleton, CI, linting, typing, and test infrastructure.

- Developer checks: fresh install, formatter/linter/type checks, unit tests, migrations up/down strategy, API schema validation, missing/invalid auth, tenant-isolation smoke tests.

- Manual gate: run service locally; create an authorized synthetic learner; submit valid/invalid events; retrieve state; restart service; verify persistence; test unauthorized access.
- Exit criteria: contracts documented; migrations reproducible; required CI green; idempotency and authorization design implemented; no unresolved P0/P1 defects.

M1 — Deterministic learner model

Implement canonical learner state, event-to-state updates, baseline mastery estimation, assisted/unassisted tracking, initial diagnosis hypotheses, rule-based recommendation policy, reason codes, state versions, and audit trail.

- Developer checks: unit-test every update rule; table-driven policy tests; property/boundary tests; integration tests against a real test database; reproducibility fixtures.
- Manual gate: test correct, incorrect, assisted, ambiguous, repeated-failure, missing-signal, and multi-concept events. Inspect both API response and persisted state.
- Exit criteria: fixed scenarios match expected results; no unsupported certainty; state changes are traceable; all core learner-state tests pass.

M2 — Closed loop with Qrie

Integrate the stable API with Qrie. Connect classroom evidence, recommendation requests, actual intervention outcomes, independent assessments, prerequisite-aware sequencing, and delayed review scheduling.

- Developer checks: API/SDK contract tests, end-to-end test flow, duplicate delivery, timeout/retry tests, graph-adapter failure, session continuity, cross-tenant tests.
- Manual gate: complete a full learning journey across at least two sessions, including an assisted attempt and a later independent assessment. Confirm that the later evidence changes state and review scheduling.
- Exit criteria: Qrie integrates without importing LAM internals; all critical flows work; errors are recoverable; manual evidence is attached to the release candidate.

M3 — Evidence-based personalization

Add strategy-outcome estimates, uncertainty calibration, proactive triggers, metacognitive evidence, session analysis/Learning Scientist output, and offline policy evaluation.

- Developer checks: sample-size safeguards, confounding-aware evaluation, confidence calibration checks, policy regression suite, session-analysis idempotency, human-readable rationale checks.
- Manual gate: inspect cases where strategy evidence is weak, conflicting, or biased by initial difficulty; verify the system reports uncertainty rather than asserting a winner.
- Exit criteria: candidate policy beats or meets predefined baseline criteria without unacceptable regressions; limitations and uncertainty are documented.

M4 — Shared intelligence

Build approved-data preparation, consent/eligibility enforcement, offline training, reproducible evaluation, model registry, staged rollout, monitoring, and rollback.

- Developer checks: data eligibility tests, leakage checks, reproducible training/evaluation, version compatibility, canary comparison, rollback and recovery tests.
- Manual gate: trace a synthetic eligible record through the governed pipeline; verify ineligible records are excluded; promote a candidate in staging; roll back; verify learner history remains intact.
- Exit criteria: no training in the request path; approval and evaluation artifacts exist; rollback is proven; candidate model meets predefined acceptance thresholds.

9. Milestone test cycle

Every milestone must use the same repeatable cycle. The exact length of a cycle depends on scope; do not equate a calendar week with a quality gate.

– Plan: link each requirement to an acceptance test and identify risks.

– Specify: write examples and expected behavior before or alongside implementation.

– Implement: add code and automated tests in the same change.

– Local verify: run formatter, linter, type checks, unit tests, relevant integration tests, and migration checks.

– Manual verify: execute the scenario checklist on the actual running build and record evidence.

– Regression: run the existing suite and verify that earlier milestone behavior remains correct.

– Review: reviewer checks architecture, contracts, privacy/security, failure modes, tests, and observability.

– Release candidate: deploy to staging, run smoke and end-to-end checks, inspect metrics and logs.

– Gate decision: pass, fail, or blocked. No ambiguous “mostly passed” status.

– Post-release: verify production/staging artifact version and monitor the defined stop conditions.

10. Performance and correctness tracking

10.1 Engineering dashboard

- CI pass rate and failed check categories.
- Test coverage by critical domain behavior; coverage percentage is diagnostic, not proof of correctness.
- Open defects by severity and age.
- Manual test pass/fail/blocked counts per milestone.
- API latency percentiles, error rate, timeout rate, and dependency failure rate.
- Event ingestion volume, duplicate-event rate, rejected-event reasons, and processing lag.
- State update conflicts, transaction failures, and idempotency violations.
- Recommendation latency, fallback rate, uncertainty distribution, and policy-version distribution.

10.2 Learning-quality dashboard

- Unassisted mastery gain per learning minute.
- Independent assessment accuracy and mastery-threshold attainment.
- Delayed retention and transfer outcomes.
- Hint dependency and assistance-level progression.
- Misconception confirmation/resolution and recurrence.
- Calibration and uncertainty metrics.
- Strategy outcomes with sample size and confidence intervals.
- Subgroup/regression analysis where data and governance permit.

Targets must be defined before release based on measured classroom needs and a documented baseline. Do not invent arbitrary latency, mastery, or improvement targets after seeing results. Report metric definitions, denominator, time window, cohort, missing-data handling, and uncertainty.

11. Release gates and rollback rules

- No merge with failing required CI checks.
- No release with open P0 or P1 defects.
- No production student data until authorization, privacy, retention, deletion, and incident controls are approved.
- No schema migration without a reviewed deployment and rollback/recovery plan.
- No model/policy promotion without a versioned evaluation report and predefined thresholds.
- No release if assisted performance can silently overwrite independent mastery.
- No release if tenant isolation, idempotency, or state consistency tests fail.
- Rollback immediately for confirmed cross-tenant leakage, unauthorized access, state corruption, major increase in errors, or materially unsafe recommendation behavior.

Rollback must include the exact artifact/policy/model version, decision owner, recovery steps, state/history implications, and post-incident review. A model rollback must not erase learner history.

12. Definition of Done

A requirement is Done only when all applicable conditions are met:

- Acceptance criteria are implemented and linked to the issue.

- Unit, integration, contract, security, and end-to-end tests are added at the appropriate level.
- Manual test record is completed for user-visible or high-risk behavior.
- Code quality gates and required CI checks pass.
- API/schema/database/policy/model version changes are documented.
- Authorization, privacy, failure handling, observability, and rollback have been reviewed.
- No open P0/P1 defects remain.
- The reviewer has approved the change and required evidence is attached.
- For learning-policy/model changes, evaluation against the baseline is complete and the result is documented.

If any applicable condition is missing, the item is not Done. It may be explicitly marked blocked or deferred with an owner and reason.

13. Required templates

13.1 Pull request checklist

- Problem and scope are clear; acceptance criteria linked.
- Design and contracts updated; compatibility considered.
- Unit/integration/contract tests added and run.
- Manual scenarios completed where required; evidence attached.
- Migrations and rollback/recovery plan reviewed.
- Auth, tenant isolation, privacy, and logging reviewed.
- Observability and failure behavior reviewed.
- CI green; no unresolved P0/P1 defects.
- Model/policy evaluation attached if behavior changes.

13.2 Manual test report template

Test ID: ____ | Requirement/Issue: ____ | Tester/date: ____ | Commit/build: ____ | Environment: ____ | Policy/model/graph version: ____

Preconditions/test data: ____

Steps: 1) ____ 2) ____ 3) ____

Expected result: ____

Actual result: ____

Status: PASS / FAIL / BLOCKED

Evidence/request IDs/log references: ____

Defect ID/severity: ____ | Retest result: ____ | Sign-off: ____

13.3 Decision-policy change record

Policy version: ____ | Owner: ____ | Motivation: ____ | Inputs and constraints: ____ | Changed rules: ____ | Benchmark dataset version: ____ | Baseline results: ____ | Candidate results: ____ | Regressions/subgroup results: ____ | Rollback version: ____ | Approval: ____

14. Final operating rule

LAM engineers are responsible for building a reliable learning system, not just endpoints that return plausible JSON. Every important behavior must have a written rule, a versioned contract, automated coverage, manual evidence where appropriate, and an owner.

Build the closed-loop deterministic baseline first. Integrate it with Qrie. Validate independent learning outcomes. Only then add more advanced personalization and shared intelligence. If the team cannot explain what evidence caused a state update or why a recommendation was selected, that behavior is not ready for production.